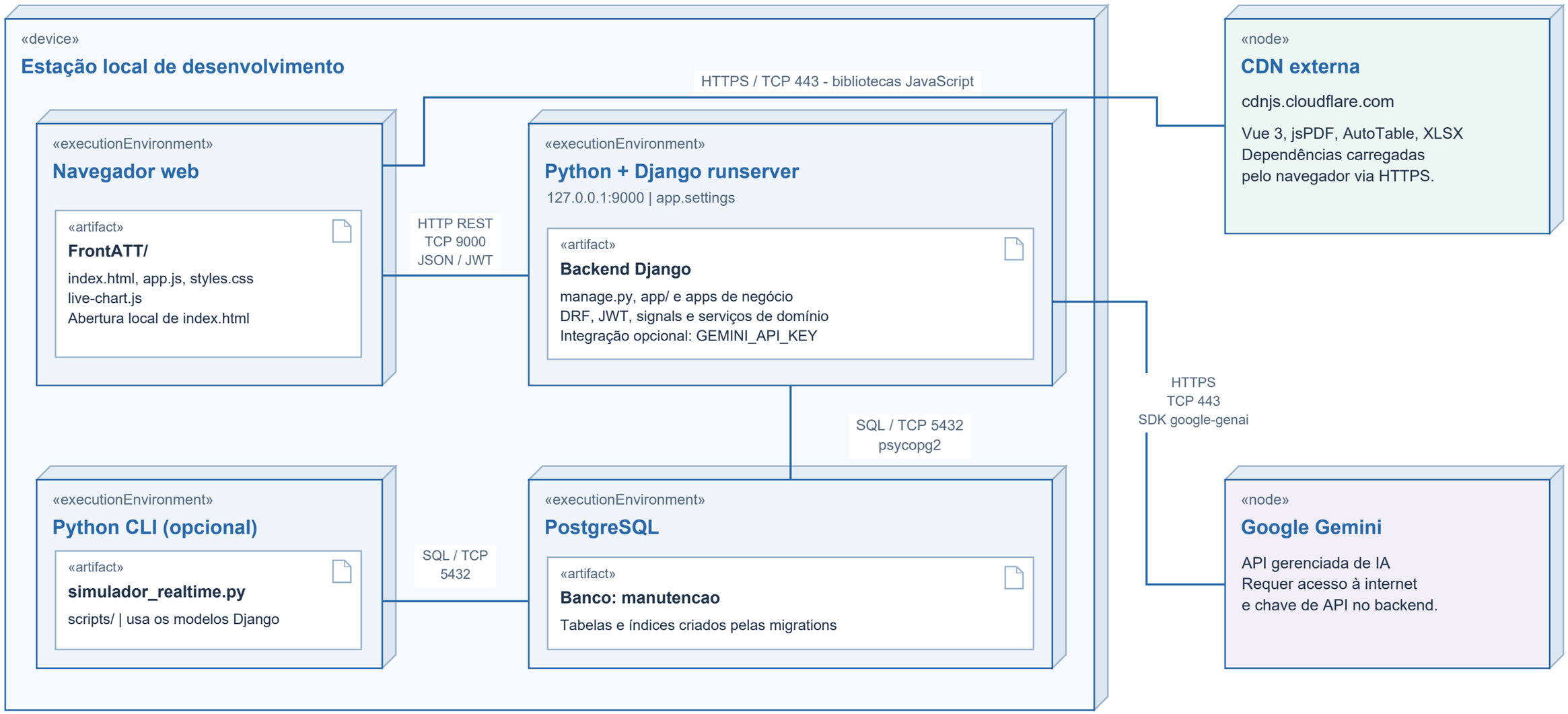
5. Diagrama de Implantação

5.1. Modelo | Cenário local documentado no repositório

NanaSmart

UML 2.5 | 11/11/2026



Caminhos de comunicação: linhas contínuas, com protocolo e porta. Os ambientes locais podem compartilhar a mesma máquina.

O simulador grava pelo ORM e aciona os signals reais. A integração com sensores físicos não está especificada no código analisado.

5.2. Infraestrutura de Execução

Nós de hardware e ambientes de execução

Estação local: o fluxo documentado usa um navegador, Python com Django e um serviço PostgreSQL em localhost. O navegador abre FrontATT/index.html; o backend é iniciado por **python manage.py runserver 127.0.0.1:9000**. O arquivo app/settings.py seleciona PostgreSQL em localhost:5432, banco manutencao.

Processo auxiliar opcional: scripts/simulador_realtime.py carrega o ambiente Django e grava leituras por modelos/ORM. Essas gravações executam os signals de telemetria. O comando manage.py analisar_tendencias pode executar a análise em lote; não há um agendador definido no repositório.

Serviços externos: o navegador carrega bibliotecas da CDN cdnjs.cloudflare.com. O backend acessa a API gerenciada Google Gemini quando há chave e recurso de IA acionado. A infraestrutura física desses fornecedores não é especificada pelo projeto.

Protocolos de rede e comunicação

Navegador ↔ backend: HTTP em TCP 9000, estilo REST, mensagens JSON e JWT Bearer nas rotas autenticadas. O frontend utiliza fetch e consulta periodicamente os dados exibidos.

Python ↔ PostgreSQL: protocolo PostgreSQL sobre TCP 5432, executando consultas SQL pelo Django ORM e psycopg2. O simulador acessa o banco por esse mesmo mecanismo, sem enviar leituras por HTTP.

Acessos externos: HTTPS em TCP 443 para carregar as bibliotecas JavaScript e para consumir Gemini por google-genai. Dentro do backend, módulos e signals se comunicam no processo Python.

Artefatos de execução implantados

Navegador: index.html, app.js, styles.css e live-chart.js de FrontATT/, além das bibliotecas CDN. **Python/Django:** manage.py, app/ e os apps accounts, authentication, ativos, manutencao, telemetria, alertas, exportacao e gemini_api, com dependências de requirements.txt. **CLI:** script de simulação e, quando executados, comandos de gerenciamento. **PostgreSQL:** tabelas, índices e relacionamentos mantidos pelas migrations.

Escopo: esta é a topologia local identificada no código e no README. O repositório não define uma topologia de produção nem integração de rede com sensores físicos; por isso, tais nós não são apresentados como implantados.